DESIGN TASK – REPORT

AI Systems Using PySpark RDDs and Intelligent Agents

1. Hybrid AI Recommendation Engine Design

Aim

To design a hybrid AI recommendation system using **PySpark RDDs** that combines content-based and collaborative filtering to provide personalized recommendations.

Introduction

Recommendation systems help users discover relevant products, movies, articles, or other items. The proposed system combines **Content-Based Filtering (CBF)** and **Collaborative Filtering (CF)**. PySpark RDDs enable large-scale user-item interaction data to be processed in parallel.

Objectives

- Process large-scale user-item interactions.
- Apply content-based filtering.
- Apply collaborative filtering.
- Combine both techniques for better recommendations.
- Monitor user behavior and feedback.
- Provide real-time personalized recommendations.

Methodology

1. Collect user profiles, item details, ratings, clicks, and likes.
2. Ingest the data using Kafka, files, or APIs.
3. Create and process RDDs using map(), filter(), join(), and reduceByKey().
4. Extract item and user features.
5. Generate content-based recommendations.
6. Generate collaborative recommendations.
7. Combine both scores using hybrid ranking.

8. Use intelligent agents to adapt recommendations.
9. Collect user feedback for continuous improvement.

Result

The system produces **Top-N personalized recommendations** based on both item similarity and user behavior.

Conclusion

The hybrid recommendation engine provides personalized and scalable recommendations by combining PySpark distributed processing with intelligent-agent-based adaptation.

---

2. Smart Disaster Detection System

Aim

To develop a distributed AI system using **PySpark and RDD operations** to detect natural disasters from satellite images, sensors, and social-media data.

Introduction

Early detection of floods, earthquakes, and other disasters can reduce damage and improve emergency response. Since disaster information comes from multiple sources, the proposed system combines and analyzes these sources using distributed processing.

Objectives

- Collect multi-source disaster data.
- Process large datasets using PySpark RDDs.
- Extract disaster-related features.
- Fuse satellite, sensor, and social-media information.
- Detect abnormal disaster conditions.
- Generate early warnings.
- Coordinate response using intelligent agents.

Methodology

1. Collect satellite images, IoT sensor data, and social-media streams.
2. Ingest the data using Kafka or Flume.

3. Process the data using PySpark RDDs.
4. Clean and transform the data.
5. Extract relevant features.
6. Fuse information based on location and time.
7. Apply disaster detection models.
8. Use Detection, Fusion, and Response Agents.
9. Generate alerts for authorities and the public.

Result

The system identifies potential disasters and generates **early warnings and response recommendations**.

Conclusion

The Smart Disaster Detection System provides fast and scalable disaster monitoring by combining multi-source data, PySpark RDD processing, data fusion, and intelligent agents.

3. AI-Based Fraud Detection Framework

Aim

To design a scalable AI-based fraud detection system using **PySpark RDDs and intelligent agents** for identifying suspicious financial transactions.

Introduction

The large number of online, card, ATM, and banking transactions makes manual fraud detection difficult. The proposed system uses distributed processing and anomaly detection to identify unusual transaction behavior.

Objectives

- Process large volumes of financial transactions.
- Clean and transform transaction data.
- Extract fraud-related features.
- Detect abnormal transaction patterns.
- Generate fraud alerts.

- Support investigation.
- Continuously improve detection accuracy.

Methodology

1. Collect financial transaction data.
2. Ingest the data using Kafka or files.
3. Convert the data into RDDs.
4. Apply map(), filter(), join(), and reduceByKey().
5. Extract features such as amount, location, time, frequency, and user behavior.
6. Apply anomaly detection.
7. Calculate transaction risk scores.
8. Use Anomaly and Alert Agents to identify suspicious transactions.
9. Use a Learning Agent to learn from confirmed fraud cases.
10. Update the model continuously.

Result

The system identifies suspicious transactions and generates **real-time fraud alerts** for investigation.

Conclusion

The framework provides scalable and adaptive fraud detection by combining PySpark distributed processing, anomaly detection, intelligent agents, and continuous feedback.

4. Autonomous Supply Chain Optimization System

Aim

To construct a distributed AI system using **PySpark** for optimizing inventory, logistics, demand forecasting, and procurement.

Introduction

Supply chains generate large amounts of sales, inventory, supplier, logistics, and market data. Efficient processing of this information is necessary for reducing costs and improving operational performance.

Objectives

- Process large-scale supply-chain data.
- Forecast product demand.
- Optimize inventory levels.
- Improve transportation routes.
- Select suitable suppliers.
- Minimize operational costs.
- Enable decentralized intelligent decision-making.

Methodology

1. Collect sales, inventory, supplier, logistics, and market data.
2. Ingest data through Kafka, files, or APIs.
3. Create RDDs and clean the data.
4. Apply map(), filter(), groupByKey(), join(), and reduceByKey().
5. Perform demand forecasting.
6. Analyze inventory requirements.
7. Optimize logistics and transportation.
8. Use Demand, Inventory, Logistics, and Procurement Agents.
9. Combine agent decisions using a Coordinator Agent.
10. Execute the optimized supply-chain plan.
11. Use performance feedback for continuous optimization.

Result

The system provides optimized **inventory levels, demand forecasts, supplier selection, and logistics decisions** while reducing unnecessary costs.

Conclusion

The autonomous supply-chain system improves efficiency and cost management by combining distributed PySpark processing with coordinated intelligent agents.

5. Personalized Learning Analytics Platform

Aim

To design an AI-driven educational platform using **PySpark RDDs and intelligent agents** to provide personalized and adaptive learning.

Introduction

Students have different learning abilities, interests, and performance levels. Traditional systems may provide the same content to all learners. The proposed platform analyzes student behavior and performance to provide personalized learning content.

Objectives

- Analyze student learning behavior.
- Process large-scale student data.
- Identify strengths and weaknesses.
- Track student progress.
- Recommend suitable learning materials.
- Provide real-time feedback.
- Adapt content difficulty.
- Support thousands of learners.

Methodology

1. Collect quiz scores, assignments, video usage, clicks, and learning activities.
2. Ingest student data through LMS systems or APIs.
3. Process data using PySpark RDDs.
4. Apply map(), filter(), flatMap(), groupByKey(), and reduceByKey().
5. Analyze student performance and behavior.
6. Identify learning gaps.
7. Use Student Profile, Content Recommendation, Feedback, and Analytics Agents.
8. Generate personalized learning content.

9. Provide real-time feedback.
10. Update the learner profile based on new performance.

Result

The platform generates **personalized learning paths, content recommendations, difficulty adjustments, and feedback** for individual students.

Conclusion

The Personalized Learning Analytics Platform provides adaptive and scalable education by combining PySpark-based distributed analytics with intelligent agents and continuous learner feedback.